

Univerzitet u Sarajevu
Elektrotehnički Fakultet
Objektno Orijentisana Analiza i Dizajn

ETF Talent Program

Sedmi projektni zadatak
Strukturalni paterni

Maj 2026.

Sadržaj

1	Strukturalni dizajn paterni — opći pregled	2
1.1	1. Adapter	2
1.2	2. Facade	2
1.3	3. Decorator	3
1.4	4. Bridge	3
1.5	5. Composite	4
1.6	6. Proxy	4
1.7	7. Flyweight	5
2	Primjena u sistemu ETF Talent Program	5
2.1	Facade — StudentServisFasada	6
2.2	Decorator — EmailServis	6
3	Ažurirani dijagram klasa	7
4	Pregled dodanih klasa	7

1 Strukturalni dizajn paterni — opći pregled

Strukturalni dizajn paterni bave se organizacijom klasa i objekata u veće, koherentne strukture. Za razliku od kreacionih paterna koji opisuju *kako* se objekti kreiraju, ili behavioralnih paterna koji opisuju *kako* objekti međusobno komuniciraju, strukturalni paterni definišu *kako se klase i objekti komponuju* kako bi se formirale fleksibilne i efikasne arhitekture.

Postoji sedam klasičnih strukturalnih paterna, definisanih u knjizi *Design Patterns: Elements of Reusable Object-Oriented Software* (Gang of Four, 1994). Svaki od njih adresira specifičnu vrstu strukturalnog problema koji se pojavljuje u razvoju objektno-orijentisanog softvera.

1.1 1. Adapter

Svrha: Konvertuje interfejs jedne klase u interfejs koji klijent očekuje. Adapter omogućava saradnju klasa čiji interfejsi nisu kompatibilni — funkcioniše kao *utikač-adapter* između dva nekompatibilna sistema.

Struktura: Postoje dva oblika — *Object Adapter* koji koristi kompoziciju (agreguje referencu na adaptiranu klasu) i *Class Adapter* koji koristi višestruko nasljeđivanje. U praksi se češće koristi Object Adapter. Adapter implementira ciljni interfejs i interno delegira pozive adaptiranoj klasi, pritom prevodeći parametre i povratne vrijednosti prema potrebi.

Kada se koristi:

- Integracija legacy koda ili vanjske biblioteke čiji interfejs ne odgovara onome koji sistem koristi
- Omogućavanje saradnje klasa koje su razvijene nezavisno i imaju nekompatibilne interfejse
- Kreiranje *wrapper* sloja između sistema i eksternih API-ja (npr. integracija payment gateway-a, biblioteke za slanje e-pošte, ili SMS servisa)

Primjer iz prakse: Aplikacija koristi interfejs `IEmailServis` s metodom `posaljiEmail()`, ali eksterna biblioteka nudi metodu `sendMail()`. Adapter klasa implementira `IEmailServis` i interno poziva `sendMail()`, čime ostatak sistema nikad ne dolazi u kontakt s vanjskom bibliotekom.

1.2 2. Facade

Svrha: Pruža jednostavan, unificirani interfejs prema složenom skupu klasa ili podsistemu. Fasada sakriva unutrašnju kompleksnost i nudi klijentima jedan ulazni punkt za interakciju s podsistemom, bez potrebe da klijenti razumiju interne detalje.

Struktura: Fasada je klasa koja agreguje reference na sve klase podsistema. Metode fasade koordiniraju pozive prema tim klasama u ispravnom redosljed i vraćaju klijentu objedinjene rezultate. Klijenti komuniciraju isključivo s fasadom, a klase podsistema ostaju međusobno nezavisne i skrivene.

Kada se koristi:

- Kada podsistem ima mnogo klasa s kojima klijenti moraju direktno komunicirati, što uzrokuje visoko sprezanje
- Kada isti složeni redosljed poziva prema više klasa mora biti ponavljan u više mjesta u kodu
- Kada se želi definisati ulazna tačka za svaki sloj arhitekture (npr. servisni sloj koji koordinira više repozitorija)
- Kada se želi smanjiti zavisnost između klijenskog koda i implementacijskih detalja podsistema

Primjer iz prakse: Sistem za narudžbe ima klase *Inventar*, *Placanje*, *Dostava* i *Notifikacija*. Umjesto da kontroler direktno koordinira sve četiri klase, fasada *NarudzbaFasada* nudi metodu *obradiNarudzbu()* koja interno koordinira sve korake ispravnim redosljed.

1.3 3. Decorator

Svrha: Dinamički dodaje nove odgovornosti objektu, bez izmjene njegove klase i bez nasljeđivanja. Dekoratori implementiraju isti interfejs kao i originalni objekt i *zamotavaju* ga, dodajući ponašanje prije ili nakon delegiranja poziva originalnom objektu.

Struktura: Zajednički interfejs (ili apstraktna klasa) definira kontrakt. Konkretna komponenta implementira osnovno ponašanje. Apstraktni dekorator implementira isti interfejs i agregiruje referencu na komponentu. Konkretni dekoratori nasljeđuju apstraktni dekorator i dodaju specifično ponašanje. Dekoratori se mogu slobodno slagati jedan na drugi (*chaining*).

Kada se koristi:

- Kada je potrebno proširiti funkcionalnost objekta na fleksibilan način koji se može kombinovati u runtime-u, bez nasljeđivanja
- Kada nasljeđivanje nije moguće (sealed klase) ili bi uzrokovalo eksploziju broja podklasa za svaku kombinaciju ponašanja
- Dodavanje logiranja, keširanja, validacije, enkripcije ili retry logike *oko* postojeće funkcionalnosti
- Kada različiti klijenti trebaju različite kombinacije funkcionalnosti iste klase

Primjer iz prakse: Osnovna klasa *TextEditor* nudi metodu *pisanje()*. Dekorator *ProvjeraSpelovanja* dodaje provjeru pravopisa, dekorator *AutoSave* dodaje automatsko čuvanje. Klijent može koristiti samo osnovu, samo jedan dekorator, ili oba u bilo kom redosljed.

1.4 4. Bridge

Svrha: Razdvaja apstrakciju od njene implementacije kako bi se obje mogle mijenjati i razvijati nezavisno. Umjesto nasljeđivanja, koristi kompoziciju —

apstrakcija agreguje referencu na implementaciju kroz interfejs.

Struktura: Apstrakcija (visoki nivo) definiše interfejs klijenta i čuva referencu na implementaciju. Implementacija (niski nivo) definiše interfejs za konkretne implementacije. Obje strane mogu imati više varijanti koje se razvijaju nezavisno.

Kada se koristi:

- Kada postoji potreba za kombinovanjem više dimenzija varijacija bez eksplozije broja podklasa — npr. M tipova apstrakcija $\times N$ implementacija = $M + N$ klasa umjesto $M \times N$
- Kada se apstrakcija i implementacija trebaju moći mijenjati nezavisno i u runtime-u
- Kada se želi sakriti implementacijske detalje od klijenata

Primjer iz prakse: Sistem za generisanje izvještaja ima vrste izvještaja (`DnevniIzvjestaj`, `MjesecniIzvjestaj`) i formate izlaza (`PDFFormat`, `ExcelFormat`). Bridge omogućava da se svaka vrsta izvještaja može kombinovati s bilo kojim formatom bez kreiranja posebne podklase za svaku kombinaciju.

1.5 5. Composite

Svrha: Komponuje objekte u stablaste strukture kako bi se hijerarhije dijel/cjelina mogle tretirati uniformno. Klijent može raditi s pojedinačnim objektima (*leaf*) i kompozicijama (*composite*) na isti način, kroz zajednički interfejs.

Struktura: Zajednički interfejs definira operacije koje važe i za listove i za kompozite. Listovi implementiraju konkretno ponašanje. Kompozitne klase sadrže kolekciju djece (koja mogu biti listovi ili drugi kompoziti) i implementiraju operacije delegiranjem prema djeci.

Kada se koristi:

- Kada se modeliraju hijerarhijske *dijel/cjelina* strukture — stabla, kategorije s podkategorijama, organizacijska stabla
- Kada klijentski kod treba raditi uniformno i s pojedinačnim objektima i s grupama objekata
- Sistem za upravljanje fajlovima (fajlovi i folderi), meniji s podmenijima, ili grafički elementi koji mogu biti grupisani

Primjer iz prakse: GUI biblioteka ima `Button` i `TextBox` kao listove, a `Panel` kao kompozit koji može sadržavati bilo koji broj komponenti, uključujući i druge panele. Operacija `render()` funkcioniše uniformno za sve.

1.6 6. Proxy

Svrha: Pruža zamjenski objekt koji kontrolira pristup originalnom objektu. Proxy implementira isti interfejs kao i originalni objekt (*Real Subject*), ali dodaje sloj logike — provjeru pristupa, logiranje, keširanje ili lazy inicijalizaciju — prije ili nakon prosljeđivanja zahtjeva.

Struktura: Interfejs definira kontrakt koji implementiraju i Proxy i Real Subject. Proxy agregiruje referencu na Real Subject i implementira isti interfejs. Klijent uvijek komunicira s interfejsom, ne znajući da li radi s Proxy-jem ili originalnim objektom.

Vrste Proxy paterna:

- **Virtual Proxy** — odgađa kreiranje skupog objekta do trenutka kada je stvarno potreban (*lazy loading*)
- **Protection Proxy** — kontrolira pristup objektu na osnovu prava korisnika
- **Remote Proxy** — predstavlja objekt koji se fizički nalazi na drugom serveru ili procesu
- **Caching Proxy** — kešira rezultate skupih operacija i vraća keširane vrijednosti pri ponovljenim zahtjevima

Primjer iz prakse: Klasa `StudentProfilProxy` implementira isti interfejs kao `StudentProfil`, ali prije vraćanja podataka provjerava da li korisnik koji zahtijeva pristup ima odgovarajuća prava (firma može vidjeti profil samo verificiranog studenta).

1.7 7. Flyweight

Svrha: Minimizira potrošnju memorije dijeljenjem zajedničkog stanja između velikog broja sitnih, sličnih objekata. Dijeli se *intrinzično* stanje (nepromjenljivo, zajedničko svim instancama), dok se *ekstrinzično* stanje (specifično za svaku instancu) prenosi kao parametar pri svakom pozivu.

Struktura: Flyweight interfejs definira metodu koja prima ekstrinzično stanje kao parametar. Konkretni Flyweight čuva intrinzično stanje. Flyweight Factory upravlja skupinom flyweight instanci i osigurava da se iste instance dijele (obično putem mape).

Kada se koristi:

- Kada aplikacija mora kreirati ogromne količine sličnih objekata koji dijele isti skup nepromjenljivih podataka
- Kada potrošnja memorije predstavlja kritičan problem zbog velikog broja instanci
- Znakovi u editoru teksta (font, veličina su intrinzično; pozicija je ekstrinzično stanje), čestice u simulaciji fizike, ikone u grafičkom sučelju

Napomena: Flyweight patern nije primjenjen u sistemu ETF Talent Program jer sistem ne kreira veliki broj sličnih objekata koji bi opravdali optimizaciju memorije ovim patern

2 Primjena u sistemu ETF Talent Program

Od navedenih sedam strukturalnih paterna, za sistem ETF Talent Program odabrana su dva: **Facade** i **Decorator**.

Facade je odabran jer sistem posjeduje složen podsistem studentskih podataka koji se sastoji od više klasa (**StudentProfil**, **AkademskiPodatak**, **RangParametri**, **Verifikacija**). Bez fasade, svaki kontroler koji treba podatke o studentu mora direktno komunicirati s ovim klasama, što uzrokuje visoko sprezanje i dupliciranje logike koordinacije.

Decorator je odabran jer klasa **EmailNotifikacija** mora podržati više ortogonalnih odgovornosti: osnovno slanje, logiranje i automatski retry. Nasljeđivanjem bi se dobila eksplozija podklasa za svaku kombinaciju, dok **Decorator** omogućava slobodno slaganje odgovornosti bez izmjene originalne klase.

2.1 Facade — StudentServisFasada

Uvodi se interfejs **IStudentServis** i klasa **StudentServisFasada** koja interno koordinira **StudentProfil**, **AkademskiPodatak**, **RangParametri** i **Verifikacija**. Kontroleri komuniciraju isključivo s **IStudentServis**, bez direktne zavisnosti o podsistemu.

IStudentServis «interface» | **StudentServisFasada**

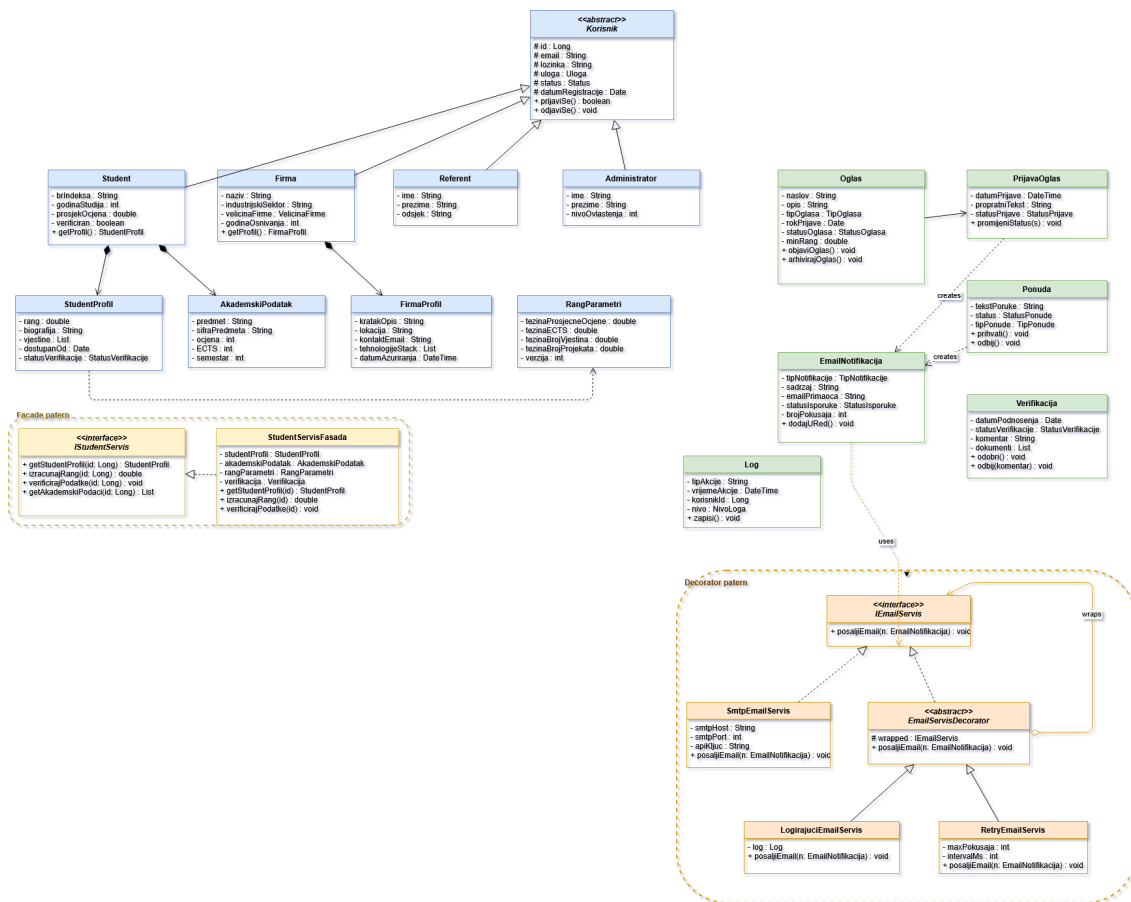
Metode: `getStudentProfil(id)`, `izracunajRang(id)`, `verificirajPodatke(id)`, `getAkademskiPodaci(id)`

2.2 Decorator — EmailServis

Uvodi se interfejs **IEmailServis** i konkretna implementacija **SmtplibEmailServis**. Apstraktni dekorator **EmailServisDecorator** omogućava slaganje konkretnih dekoratora **LogirajuciEmailServis** i **RetryEmailServis** bez izmjene postojećih klasa.

IEmailServis «interface» | **SmtplibEmailServis** | **EmailServisDecorator** «abstract» | **LogirajuciEmailServis** | **RetryEmailServis**

3 Ažurirani dijagram klasa



Slika 1: Ažurirani dijagram klasa sa dodanim strukturalnim paternima (Facade i Decorator)

4 Pregled dodanih klasa

Klasa / Interfejs	Patern	Uloga
IStudentServis	Facade	Interfejs fasade za studentski podsistem
StudentServisFasada	Facade	Koordinira StudentProfil, AkademskiPodatak, RangParametri, Verifikacija
IEmailServis	Decorator	Zajednički interfejs za sve e-mail implementacije
SmtipEmailServis	Decorator	Osnovna implementacija — šalje e-poštu putem SMTP/SendGrid
EmailServisDecorator	Decorator	Apstraktni dekorator — agregiruje IEmailServis
LogirajuciEmailServis	Decorator	Bilježi svaki pokušaj slanja u Log
RetryEmailServis	Decorator	Automatski ponavlja slanje pri grešci

Tablica 1: Pregled novih klasa uvedenih primjenom strukturalnih paterna